

Seminar Report

Team Members: Group 32

Abdur Rehman Khalid

Arnab Kumar Ghosh

Dip Chowdhury

Muhammad Umair

Pradip Kumar Das

Zihao Yang

March 1, 2026

Contents

1	Question 1: Batch vs. Streaming Processing	2
2	Question 2: Data Formats and Storage	2
3	Question 3: The Big Data Ecosystem – Is There a Logic?	2
3.1	The Volume Problem: Disk-Bound Batch Processing	2
3.2	The Random Access Problem: Wide-Column Stores	2
3.3	The Velocity Problem: The Streaming Paradigm	2
3.4	The Iteration Problem: In-Memory Unification	2
3.5	Modern Convergence: Cloud-Native and Lakehouse Architectures	3
4	Question 4: The Rise of NoSQL	3
5	Question 5: Scalability – Theory and Practice	3

1 Question 1: Batch vs. Streaming Processing

2 Question 2: Data Formats and Storage

3 Question 3: The Big Data Ecosystem – Is There a Logic?

The data engineering ecosystem is not a fragmented collection of disjointed tools, but rather a highly coherent, reactive progression. From a critical standpoint, each generation of technologies emerged explicitly to resolve the architectural bottlenecks—whether in volume, latency, or system coupling—of its immediate predecessors.

3.1 The Volume Problem: Disk-Bound Batch Processing

In the early 2000s, traditional relational databases failed to scale horizontally. Google introduced GFS and MapReduce [1] (later open-sourced as Apache Hadoop and HDFS) to solve the sheer *volume* of data using shared-nothing clusters of commodity hardware. However, Hadoop possessed a fundamental architectural flaw: aggressive disk I/O. MapReduce wrote intermediate results to physical disks between every computation phase. While highly fault-tolerant, this design rendered Hadoop completely unsuitable for low-latency querying or iterative machine learning algorithms [2].

3.2 The Random Access Problem: Wide-Column Stores

Because HDFS was strictly optimized for sequential scans of massive files, it could not serve real-time point queries. To address this limitation, Google BigTable [3] was developed as a distributed, persistent NoSQL layer over GFS, inspiring Apache HBase for the Hadoop ecosystem. Concurrently, Facebook engineered Apache Cassandra to solve HBase’s reliance on a centralized master node. Cassandra integrated BigTable’s column-family data model with Dynamo’s masterless architecture, critically trading strong consistency for extreme high availability and zero single points of failure.

3.3 The Velocity Problem: The Streaming Paradigm

As data velocity increased, the latency of batch processing became a critical liability. Twitter Storm pioneered distributed stream processing but exhibited severe centralized bottlenecks. Twitter Heron was a direct, critical redesign of Storm, decoupling the scheduling framework to isolate task resources. Meanwhile, to prevent high-velocity data from overwhelming these processing engines, LinkedIn introduced Apache Kafka [4]. Kafka acted as a distributed, log-structured commit broker, successfully decoupling data ingestion from computation.

3.4 The Iteration Problem: In-Memory Unification

Addressing Hadoop’s disk-I/O bottleneck directly, Apache Spark [2] introduced Resilient Distributed Datasets (RDDs), enabling fault-tolerant, *in-memory* processing that accelerated iterative algorithms by orders of magnitude. Spark also attempted to unify the fragmented batch and streaming ecosystems via “micro-batching.” However, this abstraction inherently injected latency into streaming pipelines. In a critical counter-paradigm,

Apache Flink emerged as a native stateful stream processor, conceptually modeling batch processing simply as a bounded stream to achieve true low-latency processing.

3.5 Modern Convergence: Cloud-Native and Lakehouse Architectures

Today, the ecosystem rejects the rigid coupling of storage and compute that defined the Hadoop era. Modern architectures leverage cheap, highly scalable cloud object storage decoupled from elastic compute engines. This evolution has culminated in the “Data Lakehouse” [5]—powered by open table formats like Apache Iceberg. The Lakehouse directly addresses the historical dichotomy between chaotic data lakes and expensive data warehouses, bringing ACID transactions and schema enforcement directly to decoupled cloud storage.

4 Question 4: The Rise of NoSQL

5 Question 5: Scalability – Theory and Practice

References

- [1] J. Dean and S. Ghemawat, “Mapreduce: Simplified data processing on large clusters,” in *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation*. USENIX Association, 2004, pp. 10–10.
- [2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*. USENIX Association, 2012, pp. 2–2.
- [3] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A distributed storage system for structured data,” in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*. USENIX Association, 2006, pp. 205–218.
- [4] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, vol. 11, 2011, pp. 1–7.
- [5] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)*, 2021.